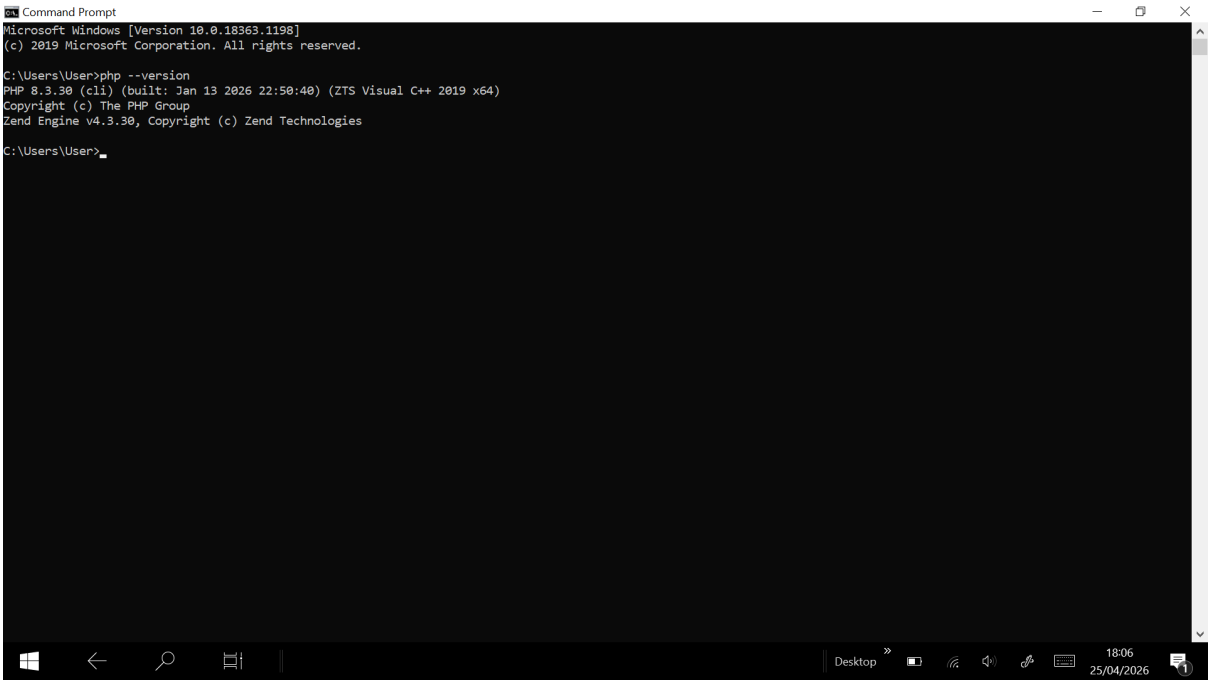
Nama : Talitha Puspitasari
NIM : 242410101017
Kelas : PWEB A

AKTIVITAS MANDIRI 1 — Instalasi & Setup Proyek Laravel

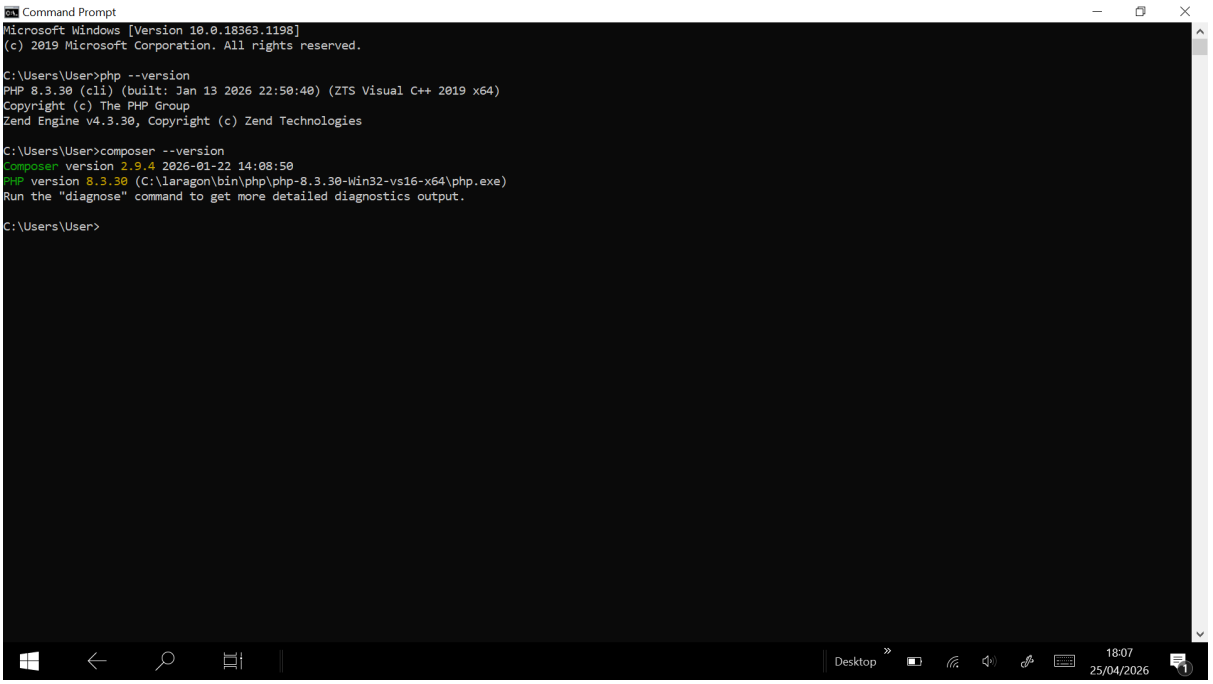
1. Instal XAMPP terbaru, pastikan PHP 8.2+ tersedia (cek: php --version di terminal)
2. Instal Composer dari getcomposer.org, verifikasi dengan: composer --version
3. Buat proyek Laravel baru: composer create-project laravel/laravel siak-app
4. Buat database baru 'siak_db' di phpMyAdmin
5. Konfigurasi file .env: isi DB_DATABASE, DB_USERNAME, DB_PASSWORD sesuai lokal
6. Jalankan: php artisan key:generate
7. Jalankan server: php artisan serve — buka http://127.0.0.1:8000 di browser
8. Eksplorasi struktur folder, buka setiap folder dan pahami fungsinya
9. Jalankan php artisan route:list dan catat route apa saja yang sudah terdaftar

Tips: Gunakan Laragon (laragon.net) sebagai alternatif XAMPP — lebih ringan dan mendukung Laravel secara native dengan virtual host otomatis.

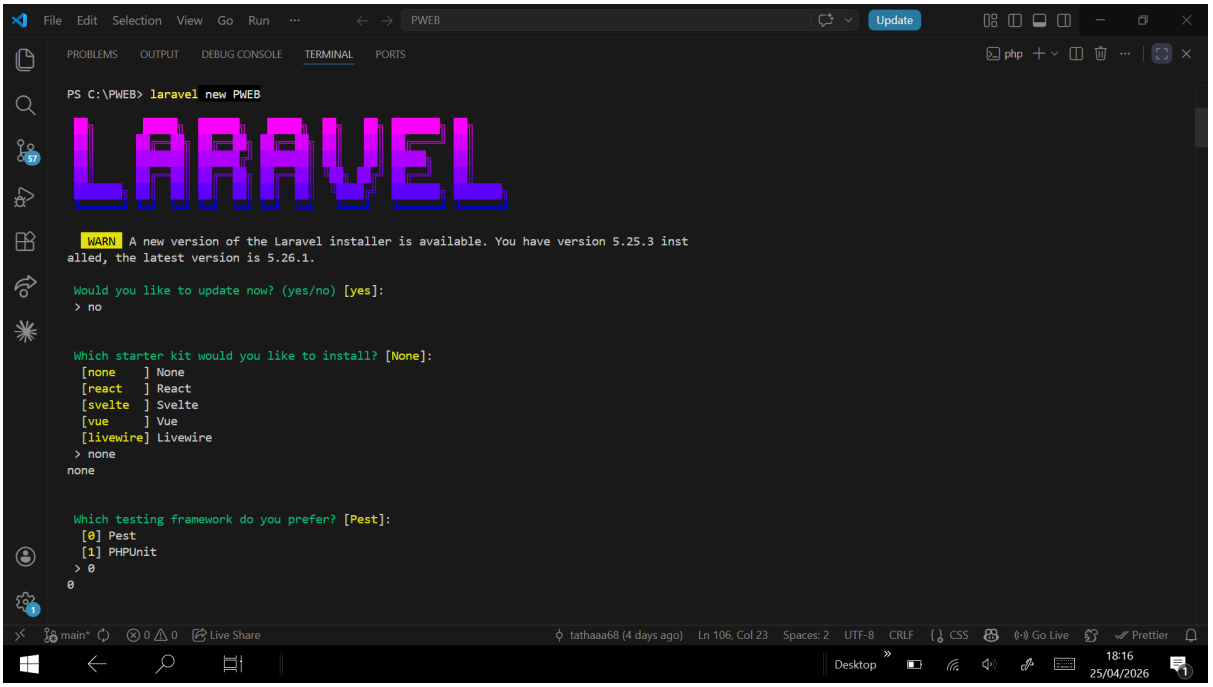
1. Instal XAMPP terbaru, pastikan PHP 8.2+ tersedia (cek: php --version di terminal)

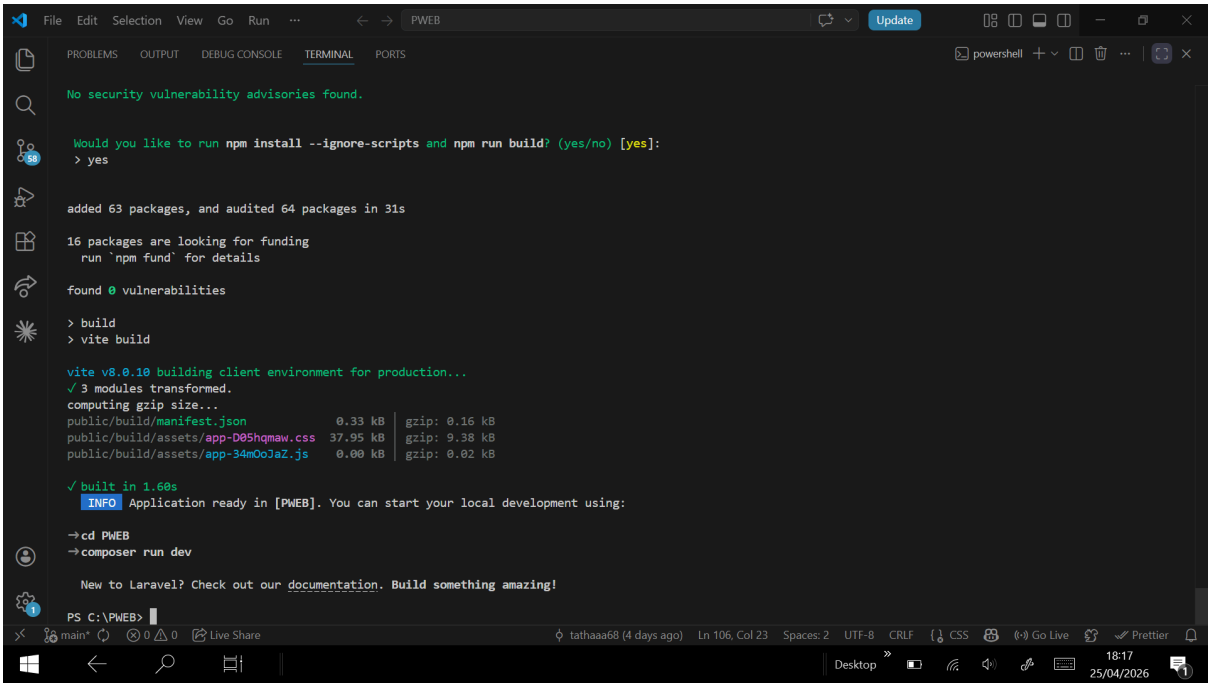


2. Instal Composer dari getcomposer.org, verifikasi dengan: composer --version

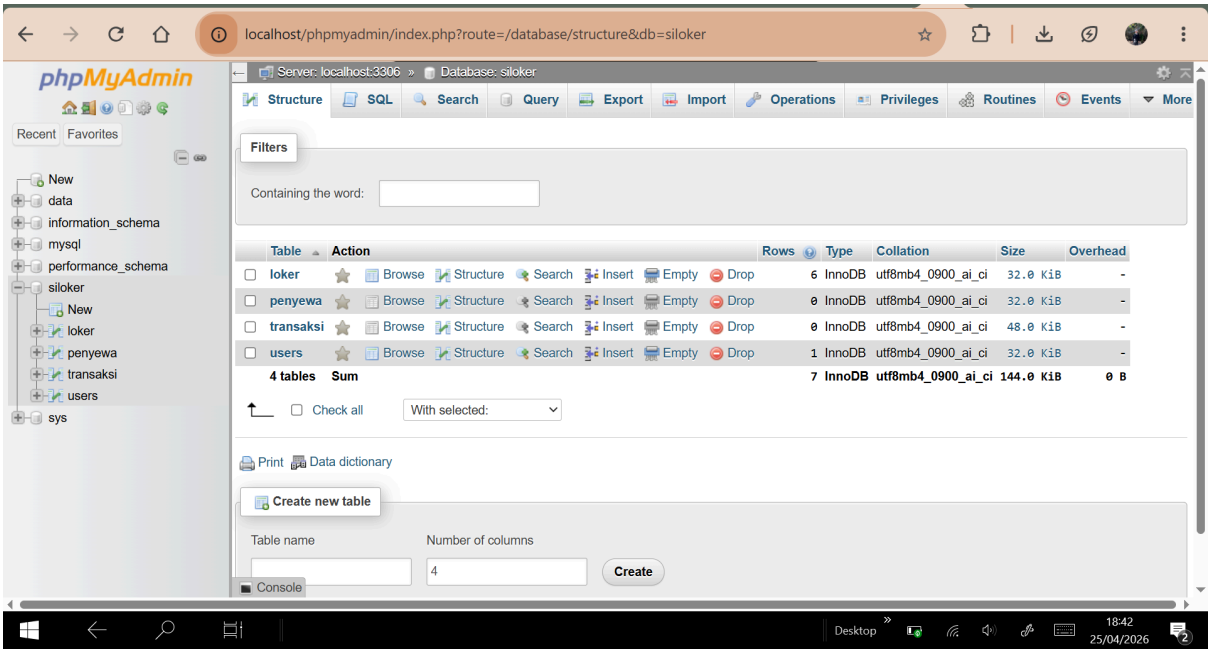


3. Buat proyek Laravel baru: composer create-project laravel/laravel siak-app

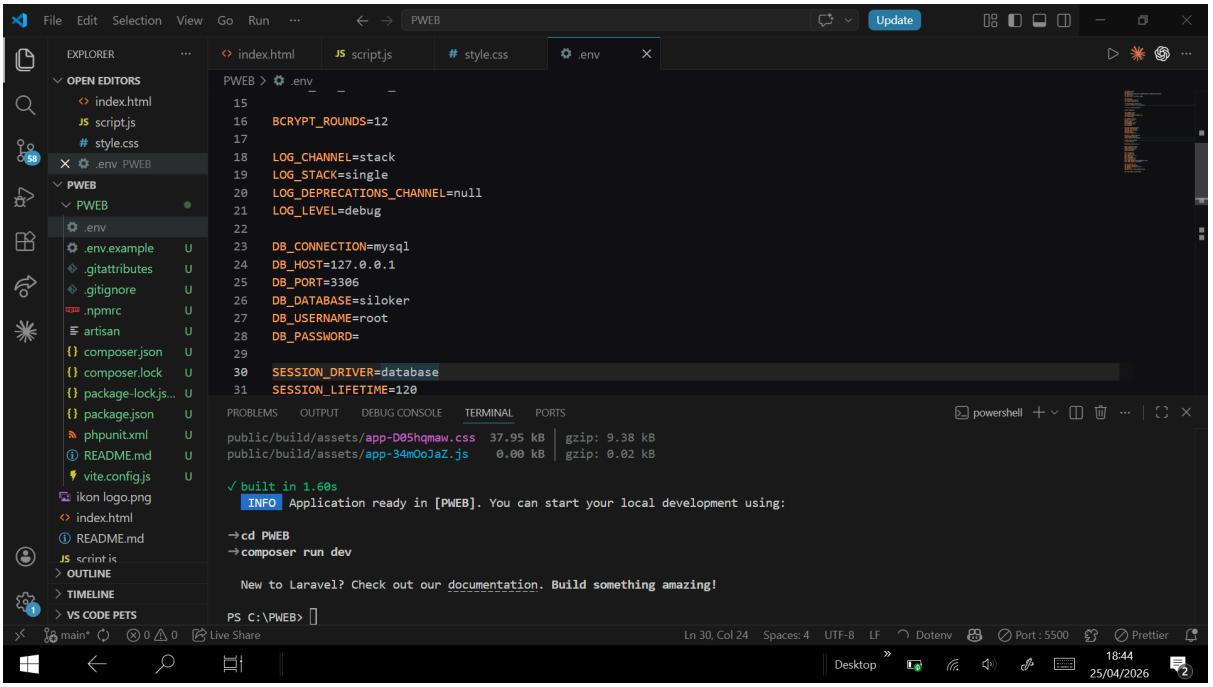




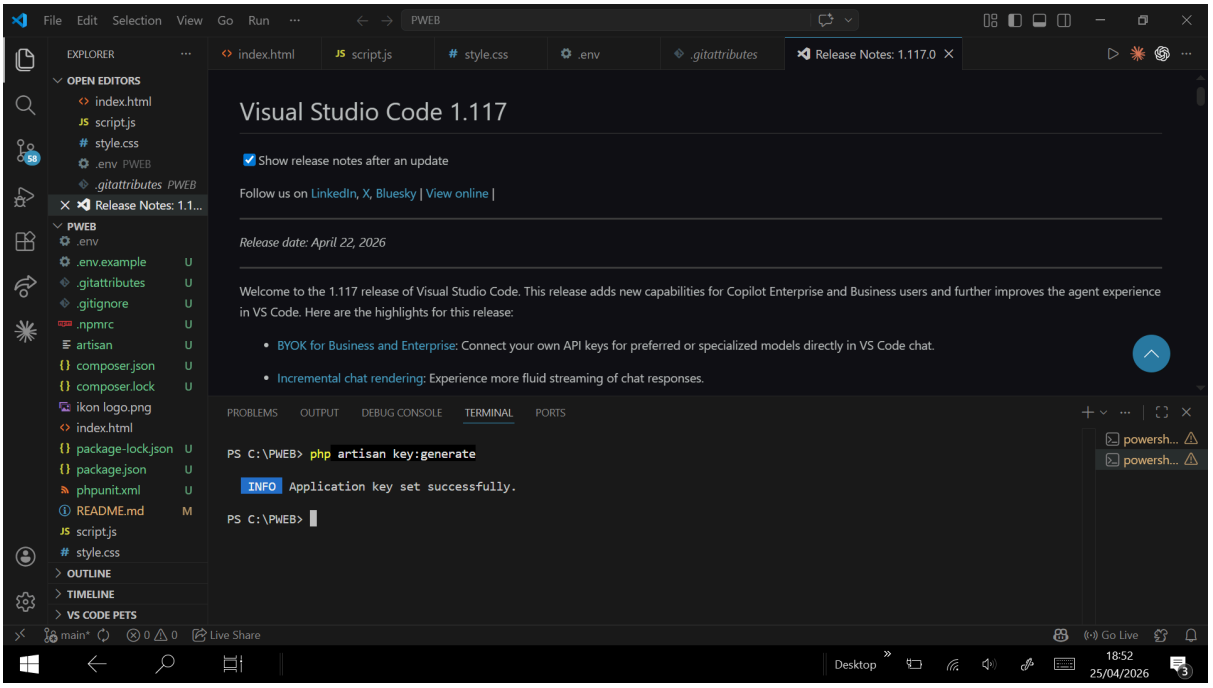
4. Buat database baru 'siak_db' di phpMyAdmin



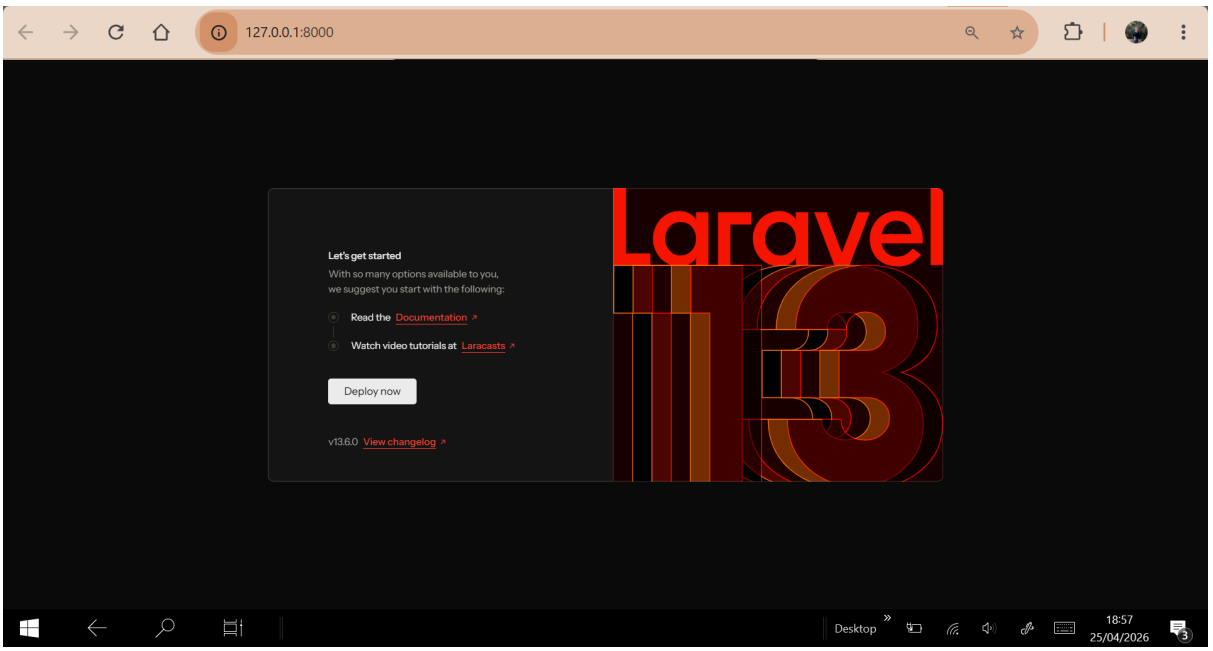
5. Konfigurasi file .env: isi DB_DATABASE, DB_USERNAME, DB_PASSWORD sesuai lokal

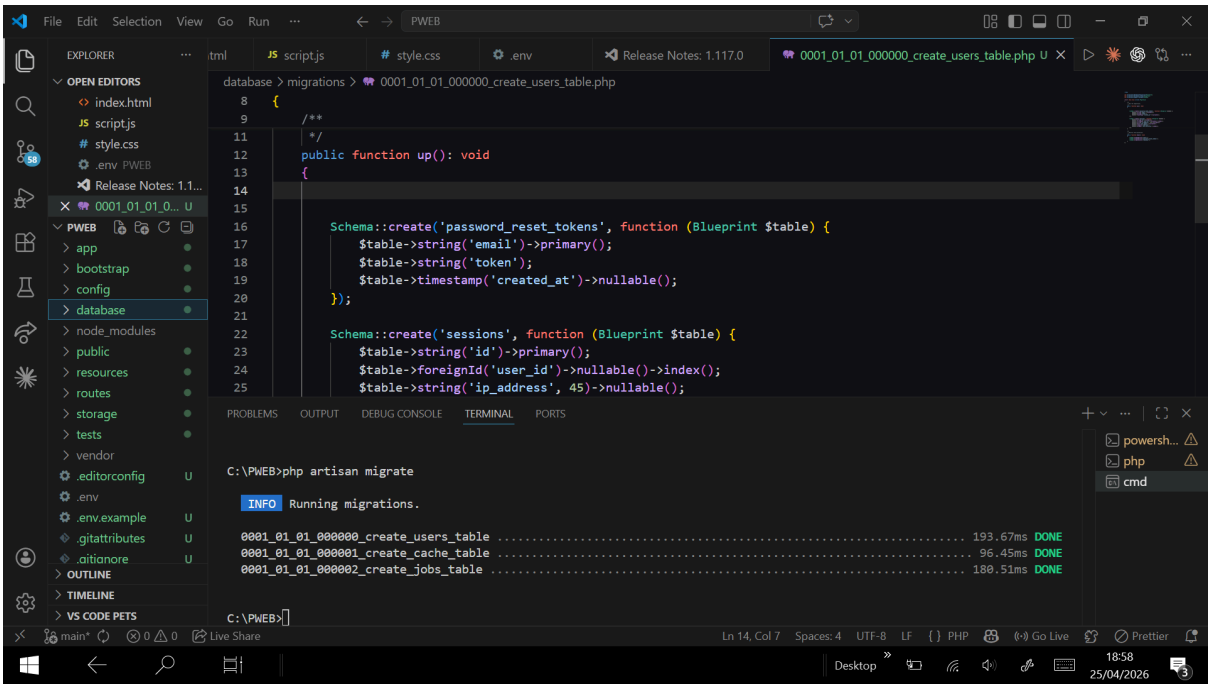


6. Jalankan: php artisan key:generate



7. Jalankan server: php artisan serve — buka http://127.0.0.1:8000 di browser





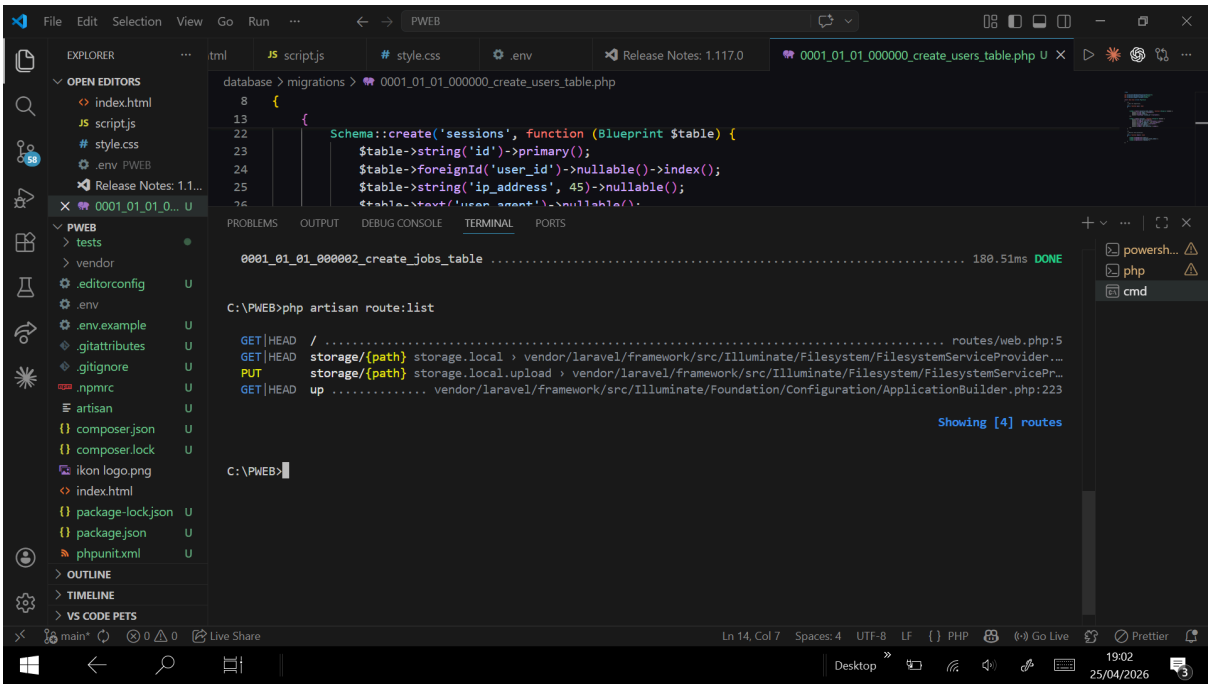
8. Eksplorasi struktur folder, buka setiap folder dan pahami fungsinya

No	Bagian	Folder/File	Fungsi
1	Folder Inti Aplikasi	app/	Merupakan pusat logika bisnis aplikasi yang mengatur keseluruhan proses utama.
		app/Http/Controllers/	Menyimpan file Controller yang berfungsi untuk memproses permintaan (request) dari pengguna dan mengatur alur respon.

		app/Models/	Berisi Model yang digunakan untuk berinteraksi langsung dengan tabel dalam database.
		app/Providers/	Digunakan untuk melakukan konfigurasi dan registrasi layanan dalam sistem Laravel.
		bootstrap/	Berisi file untuk proses inisialisasi awal aplikasi, seperti pengaturan cache agar performa lebih optimal.
2	Konfigurasi & Lingkungan	config/	Menyimpan berbagai file konfigurasi aplikasi seperti database, mail, dan sistem file.
		.env	File yang berisi variabel lingkungan yang bersifat sensitif, seperti kredensial database.
3	Database & Migrasi	database/	Direktori utama yang berkaitan dengan pengelolaan database.
		database/migrations/	Digunakan untuk mendefinisikan struktur tabel database menggunakan kode PHP.
		database/seeders/ & factories/	Digunakan untuk mengisi database dengan data awal atau data contoh secara otomatis.
4	Folder Publik & Aset	public/	Satu-satunya folder yang dapat diakses langsung oleh browser sebagai pintu masuk aplikasi.
		public/index.php	File utama (entry point) yang pertama kali dijalankan saat aplikasi diakses.
		public/assets (gambar, CSS, JS)	Menyimpan file statis seperti gambar, stylesheet, dan JavaScript hasil kompilasi.
5	Frontend & Tampilan	resources/	Menyimpan berbagai file yang berkaitan dengan tampilan antarmuka pengguna.
		resources/views/	Berisi file Blade (.blade.php) yang digunakan untuk menyusun struktur HTML aplikasi.

		resources/css/ & js/	Menyimpan file mentah untuk styling dan script sebelum diproses oleh tools seperti Vite atau Mix.
6	Rute & Navigasi	routes/	Mengatur navigasi dan endpoint aplikasi.
		routes/web.php	Digunakan untuk mendefinisikan rute berbasis web (halaman yang diakses pengguna).
		routes/api.php	Digunakan untuk mendefinisikan endpoint API.
7	Manajemen Paket & Penyimpanan	node_modules/	Berisi paket JavaScript yang diinstal melalui npm.
		vendor/	Berisi library PHP dan framework Laravel yang diinstal melalui Composer.
		storage/	Digunakan untuk menyimpan file hasil proses aplikasi seperti log, cache, dan file unggahan pengguna.

9. Jalankan php artisan route:list dan catat route apa saja yang sudah terdaftar

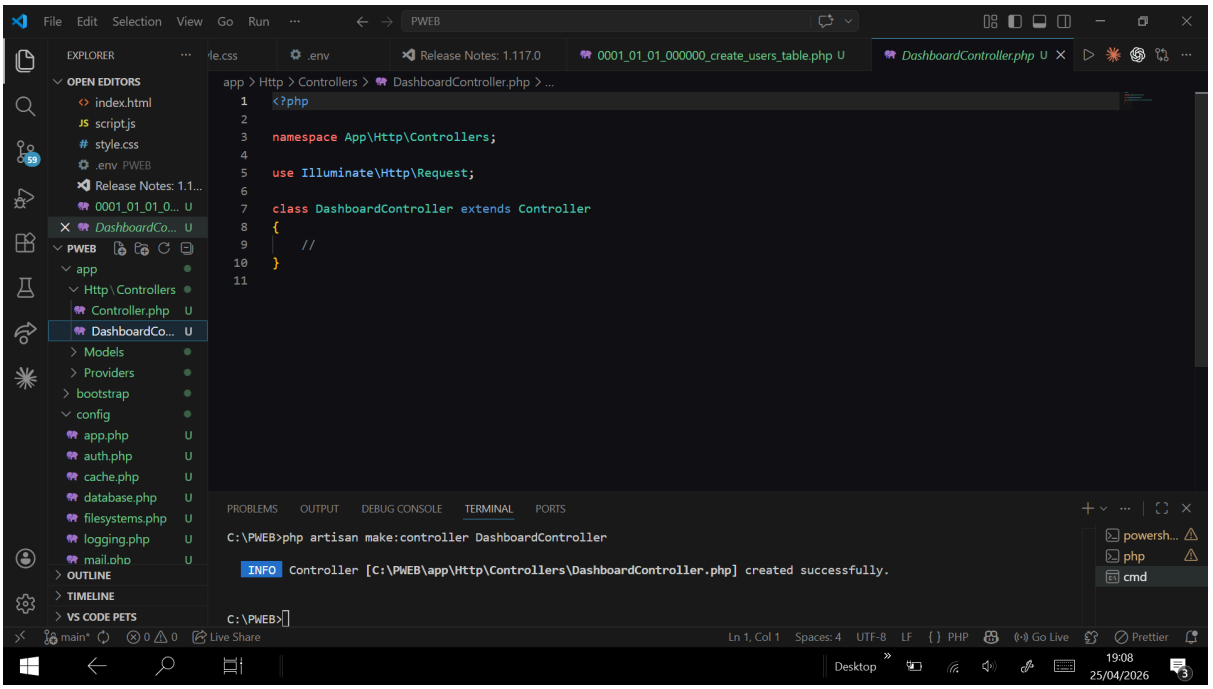


AKTIVITAS MANDIRI 2 — Membuat Routing & Controller Dashboard

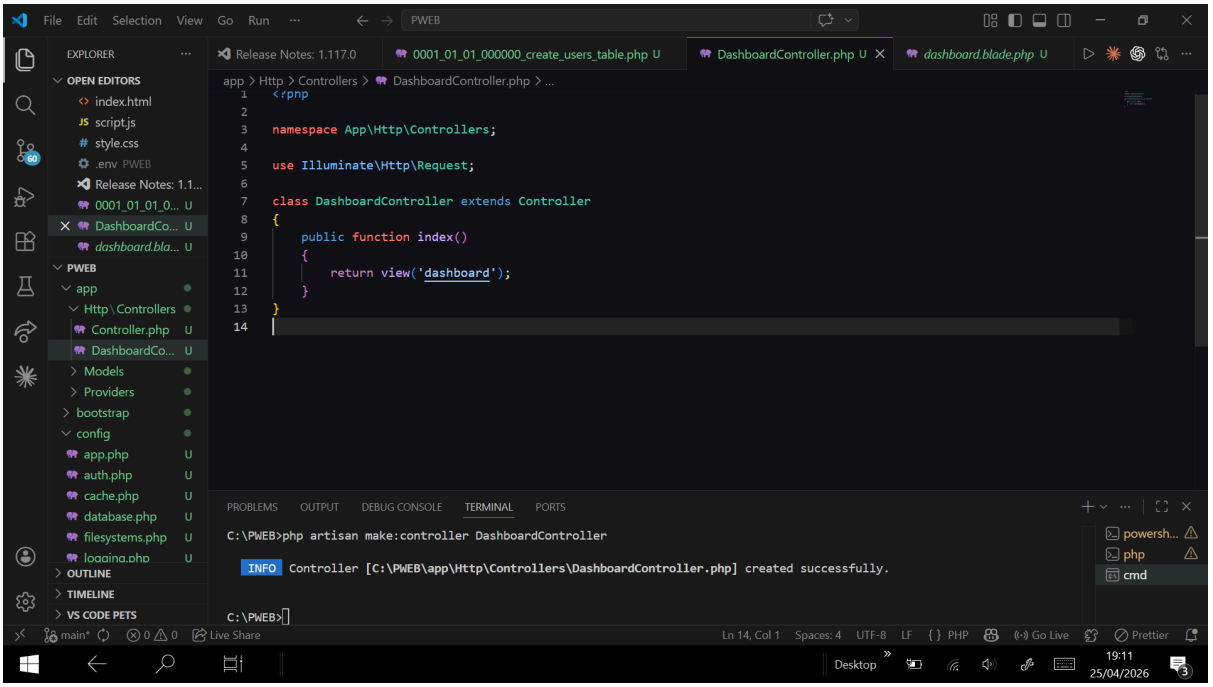
1. Buat Controller DashboardController: php artisan make:controller DashboardController
2. Tambahkan method index() yang mengembalikan view('dashboard') di DashboardController
3. Daftarkan route di routes/web.php: Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard')
4. Buat file view sementara resources/views/dashboard.blade.php berisi '<h1>Dashboard SIAK</h1>'
5. Akses http://127.0.0.1:8000/dashboard di browser — pastikan tampil
6. Buat route /tentang dengan Route::view('/tentang', 'tentang') dan buat view-nya
7. Jalankan php artisan route:list — amati semua route yang terdaftar
8. Coba buat Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b) dan akses /hitung/10/25

Tips: Route::view() adalah shortcut untuk route yang hanya menampilkan view tanpa logika. Sangat berguna untuk halaman statis seperti 'Tentang Kami'.

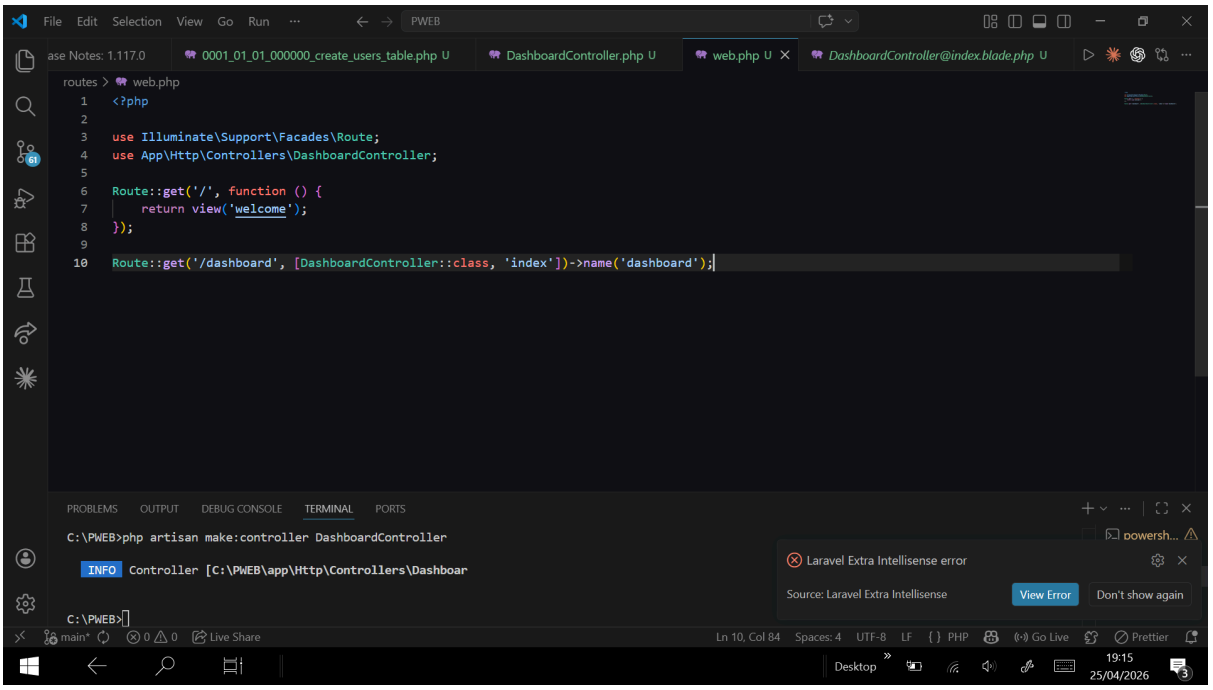
1. Buat Controller DashboardController: php artisan make:controller DashboardController



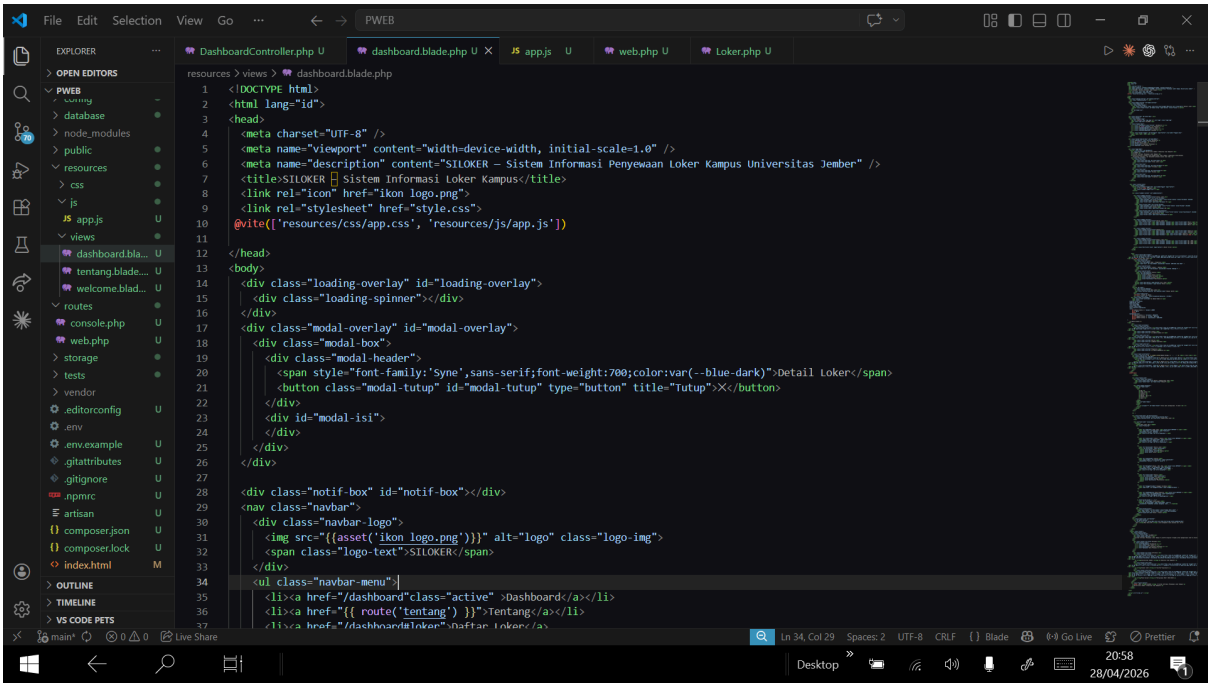
2. Tambahkan method index() yang mengembalikan view('dashboard') di DashboardController



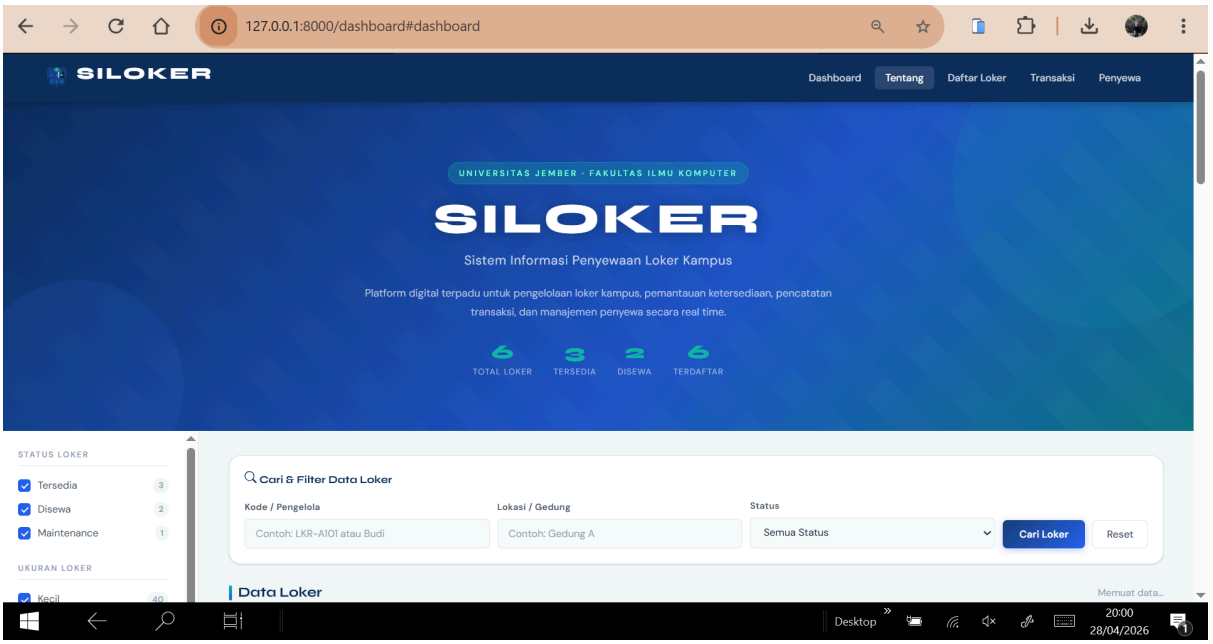
3. Daftarkan route di routes/web.php: Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard')



4. Buat file view sementara resources/views/dashboard.blade.php berisi '<h1>Dashboard SIAK</h1>'

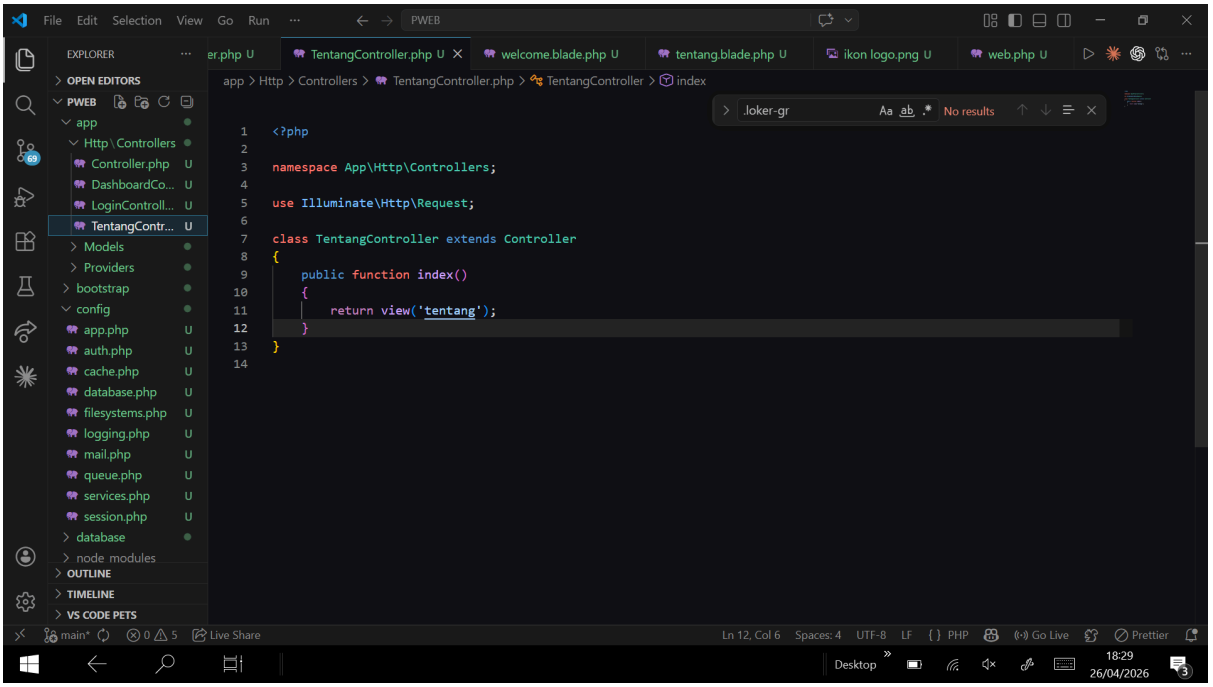


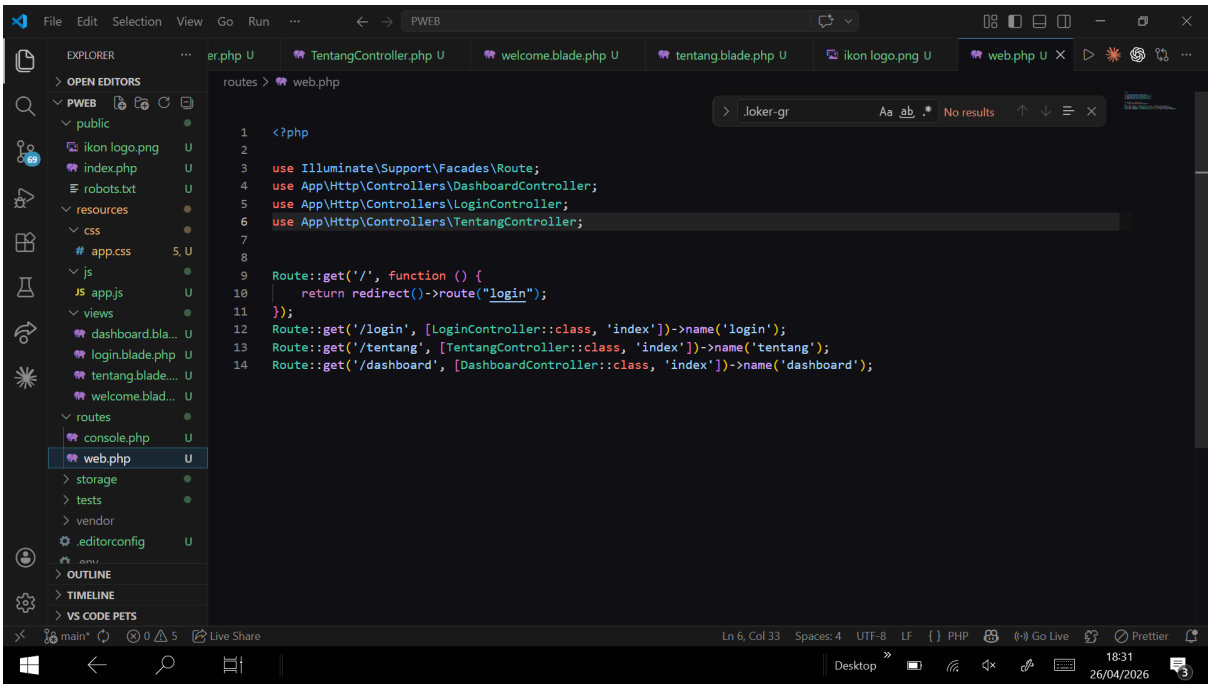
5. Akses http://127.0.0.1:8000/dashboard di browser — pastikan tampil



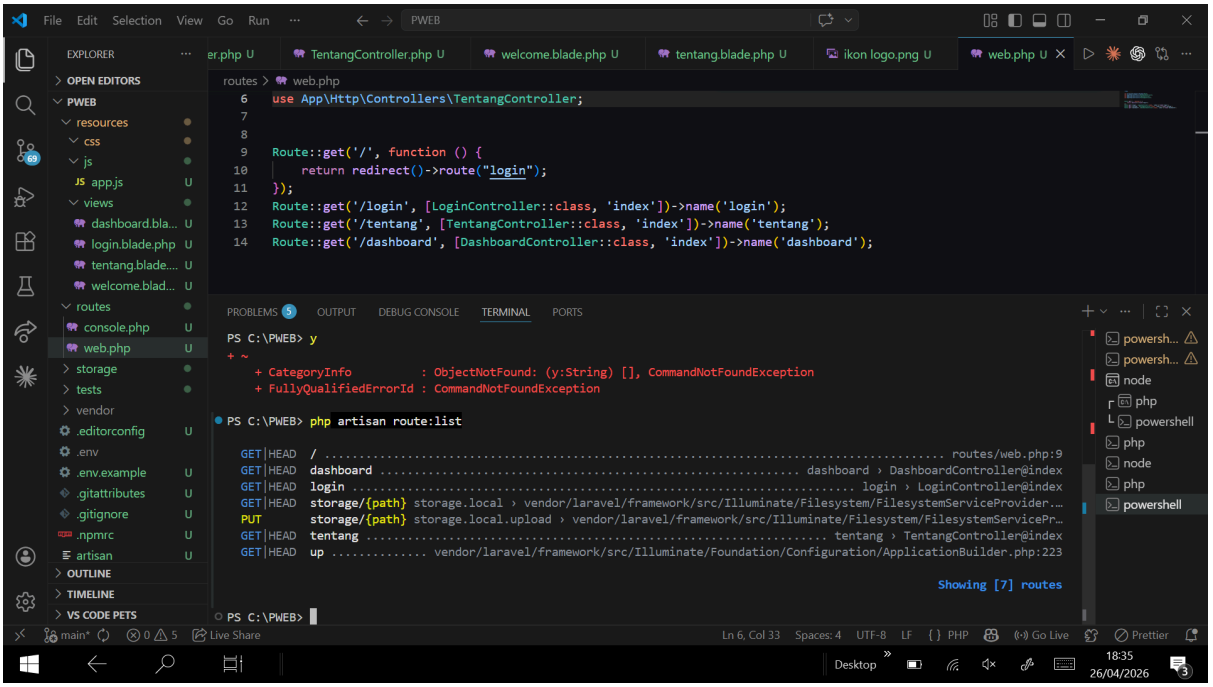
Penjelasan : disini saya menggunakan tampilan projek saya

6. Buat route /tentang dengan Route::view('/tentang', 'tentang') dan buat view-nya





7. Jalankan php artisan route:list — amati semua route yang terdaftar



8. Coba buat Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b) dan akses /hitung/10/25